

Ansible project in project book

classmate

Date _____
Page _____

Project Goal:-

→ To learn how ansible automates server management.

- Instead of logging into every server & running commands manually, we use one controller machine to manage multiple servers.

Laptop (PowerShell)

↓
SSH using pem

Controller EC2 (Ubuntu)
Ansible installed

SSH using public key

↓
managed Node (Ubuntu)

No ansible installed

so, we created 2 EC2 instances

controlled
Node

managed
Node

Purpose for this:-

i] Controller can

- run Ansible
- store playbooks
- controls other servers

ii] managed Node receives instructions

Configure security groups:-

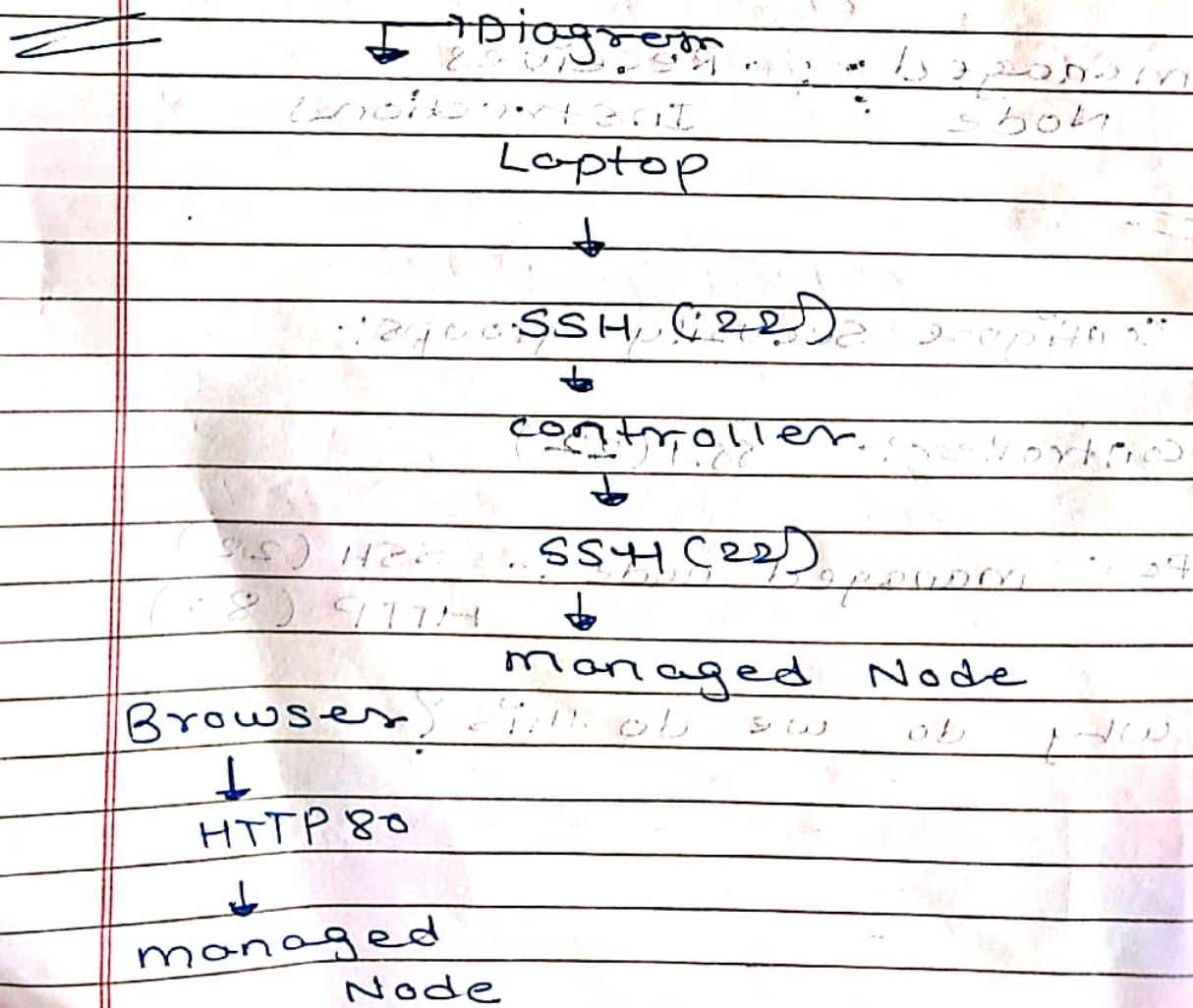
Controller:- SSH (22)

For managed node :- SSH (22)
HTTP (80)

Why do we do this?

Purpose is :-

- i] Laptop can connect through SSH command into Eec2 controller.
- ii] controller can access managed node through SSH 22.
- iii] Browser can have access to managed through HTTP 80



we connect to controller using:-

- `ssh -i [file location address of .pem] ubuntu@Public-IP`

- we install ansible using

`sudo apt install ansible`

why do we do this

without
ansible

↓
Login
Server 1
(run command)

↓
Login Server 2
(run command)

↓
repeat until
all servers

with
ansible

Instead of repeating work on every server, one command manages all server.

1. = Generate SSH key.

so by using `ssh-keygen` de.
we get
• pub. & private key. For
controller.

and our task for now is
passwordless communication

↓
So we copy public key
of controller node &
paste it in authorized
keys

↓
authorized key folder is
in `~/.ssh/authorized-keys`

↓
It tells your managed node
that we trust this
controller.

lets Test SSH.

ssh ubuntu@PrivateIP

→ ~~It~~ ~~ssh~~ Controller should connect to managed node.

lets install Ansible only on controller.

— This will control everything just by controller

— managed nodes will receive instructions.

create inventory. → why?

↓
Because inventory tells ansible which servers to manage.

eg:- in inventory:-

[web servers]

10.0.137.225

ansibleuser-ubuntu.

↓
we write this so ansible gets to know what user it is & not guess
(eg:- Ubuntu, windows, RH)

So when we have groups we run commands like

- `ansible webserver -i inventory -m ping`

- or running a playbook:-

`ansible-playbook -i inventory install nginx.yml`

Inside the playbook:-

`hosts: webserver`

Create a playbook

file as:- `install-nginx.yml`

where we will be doing:-

Install Nginx

Start Nginx

Enable Nginx

Run playbook!:-

= ansible-playbook -i inventory
install-nginx.yml

Lost overall workflow

Creates EC2 instances!:-

↓
Connect to controller

↓
Install Ansible

↓
Generate SSH key

↓
Copy public key

↓
Passwordless SSH

↓
Create Inventory

↓
Ansible Ping

↓
Create playbook

↓
Run playbook

↓
Managed Node Configured

↓
Verified in Browser.